



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



Cloud Computing Architecture

Semester project report

Group 095

Jeferson Morales Mariciano - 21-980-446

Athena Wang - 21-945-894

Marita Berger - 21-949-979

Systems Group
Department of Computer Science
ETH Zurich
May 16, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.
Divergence from the template can lead to subtraction of points on graded parts of the project.
- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.
- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.
- **Use of AI Tools:** The use of AI tools must adhere to [ETH regulations](#). **You take responsibility for the content you submit.** You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 3 [68 points]

1. [34 points] Describe and analyze your hand-crafted policy.
 - (a) [17 points] With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time ¹ across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency > 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes	35.3	1.2
blackscholes	41.3	1.5
canneal	75.0	1.0
freqmine	92.7	3.1
radix	11.0	1.7
streamcluster	141.0	1.0
vips	52.0	1.7
total time	219.0	2.0

Answer: The SLO violation ratio is 0 for all three runs. The p95 latency stays well below 1ms throughout, peaking at 764.9 μ s in run 1, 546.0 μ s in run 2, and 623.0 μ s in run 3.

Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperrf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in **the color of vips** for core x from when vips is started to when it ends if vips ran on core x .

Plots: Plots are given in 3, 4 and 5.

- (b) [17 points] Describe and justify the “optimal” scheduling policy you have designed.
 - Which node does memcached run on? Why?

Answer: Memcached runs on **node-a-8core**. The choice of node-a-8core is motivated by the fact that it provides 6 spare cores (2–7) for batch jobs after reserving cores 0–1 for memcached. This is the only node with enough remaining capacity to host the more thread-hungry batch jobs while keeping them strictly separated from

¹Here, you should only consider the runtime, excluding time spans during which the container is paused.

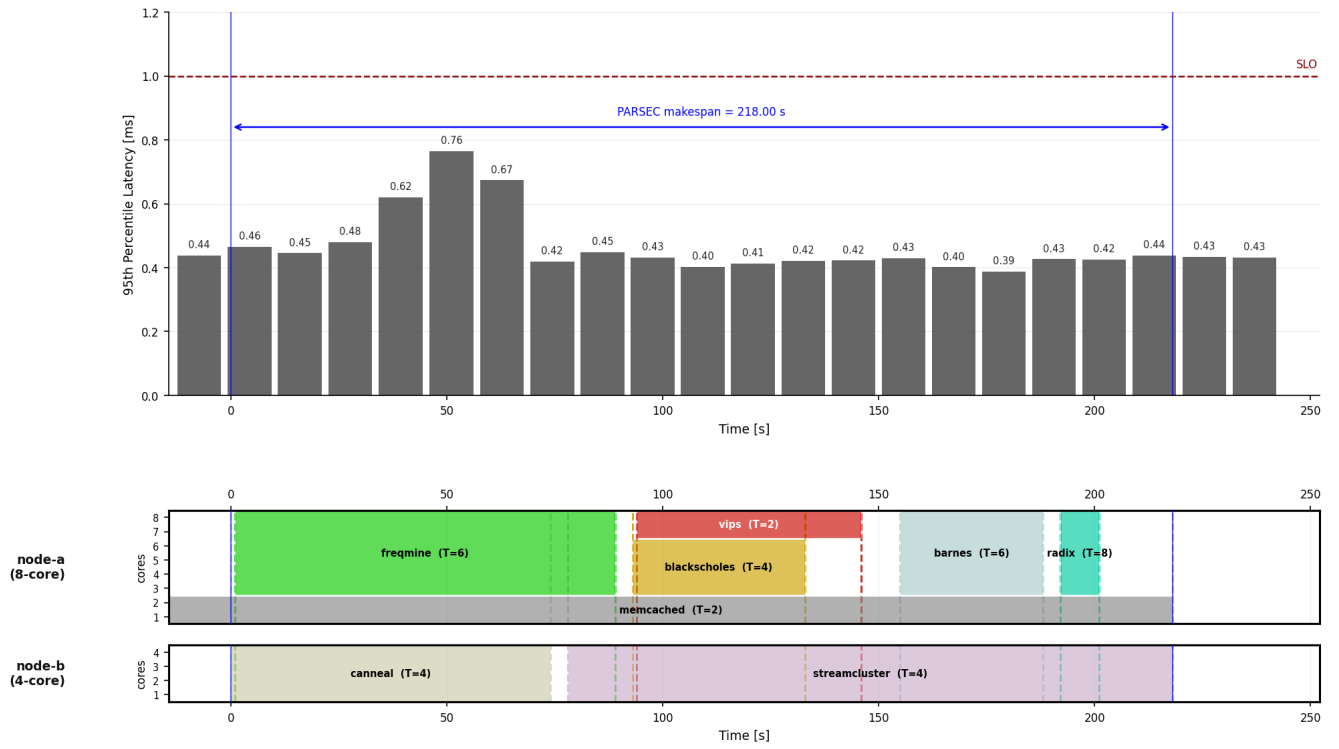


Figure 3: Memcached p95 latency over time (top) and concurrent/colocated jobs of run 1 (bottom)

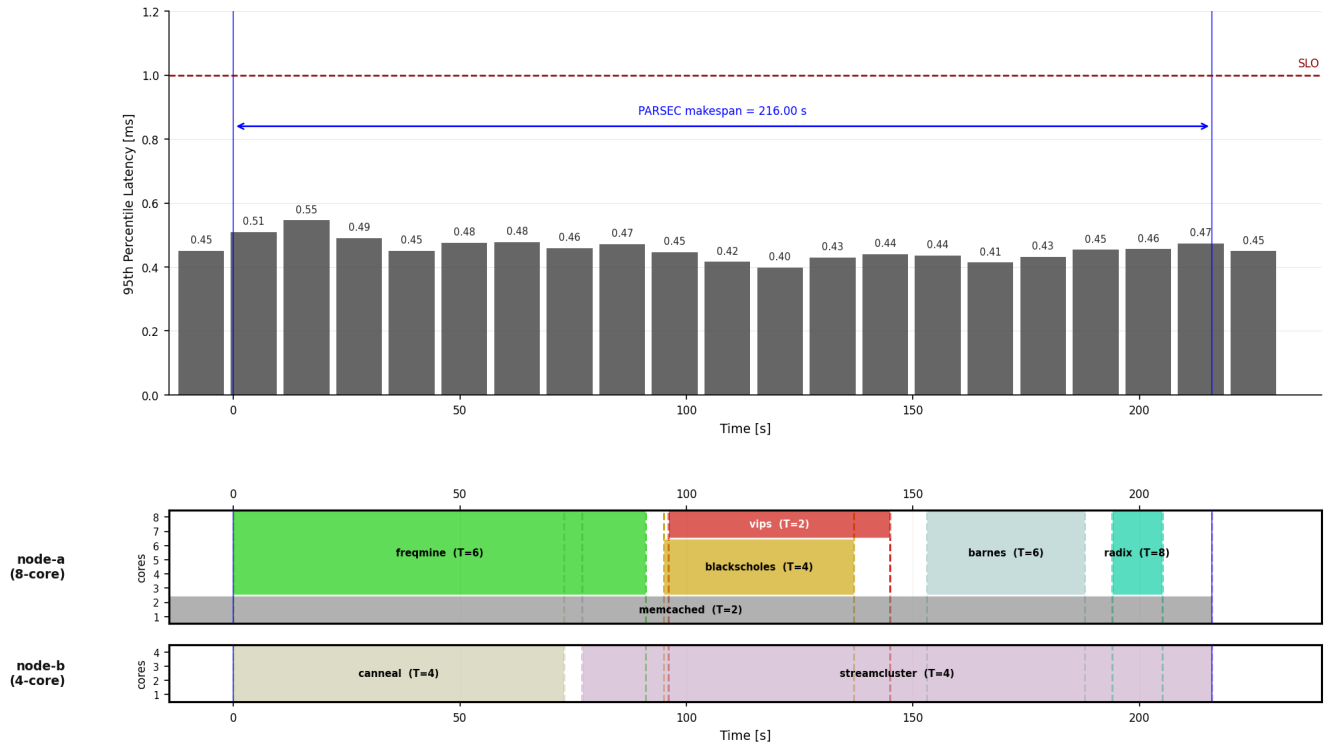


Figure 4: Memcached p95 latency over time (top) and concurrent/colocated jobs of run 2 (bottom)

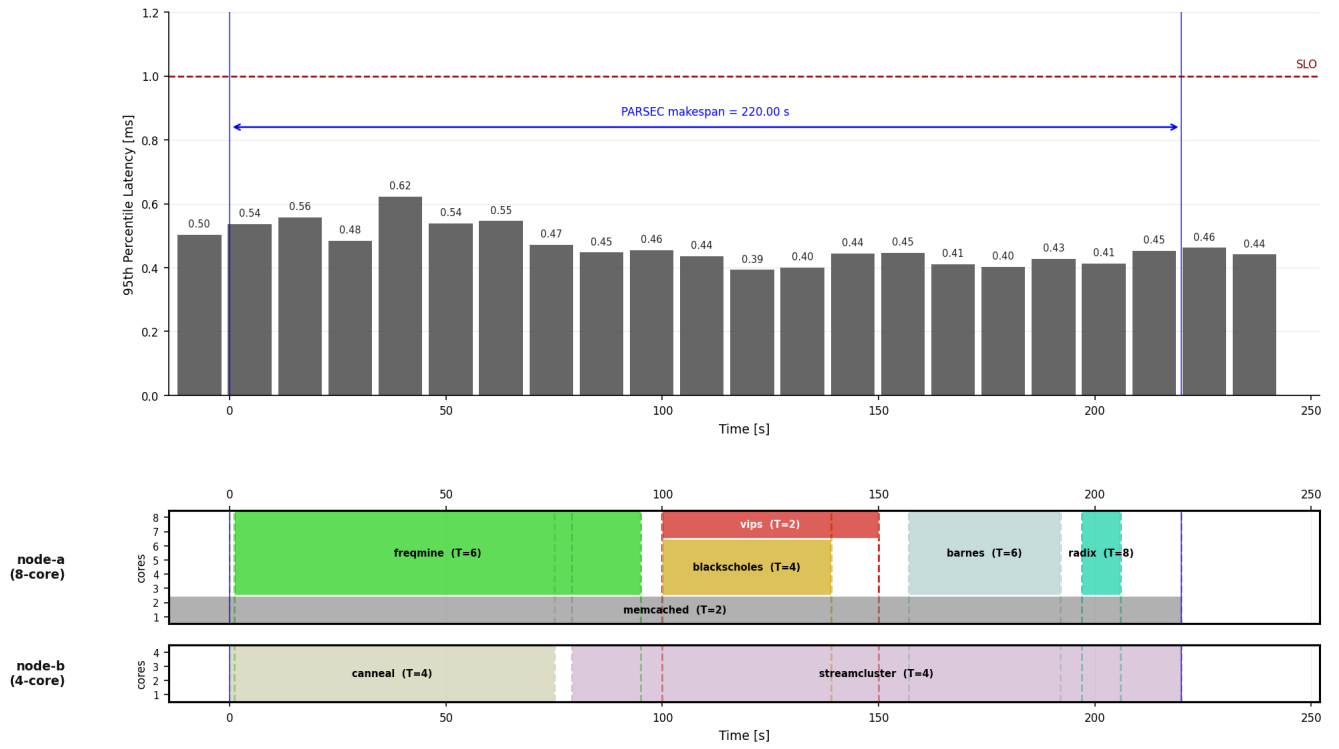


Figure 5: Memcached p95 latency over time (top) and concurrent/colocated jobs of run 3 (bottom)

memcached via non-overlapping core assignment. Placing memcached on node-b-4core instead would leave only 2 cores available for batch jobs, limiting parallelism and increasing the makespan. Apart from that, node-b-4core has less memory bandwidth and smaller caches than the 8-core node, which affect memcached's memory-latency-sensitive nature under 30K QPS.

- Which node does each of the 7 batch jobs run on? Why?

Answer:

- **barnes**: **node-a-8core**. Barnes is assigned 6 cores, utilizing all spare cores on node-a after reserving cores 0-1 for memcached. This maximizes resource utilization and avoids leaving cores 6-7 idle, which was the case in earlier policy iterations.
- **blackscholes**: **node-a-8core**. Blackscholes is assigned 4 cores. It is co-scheduled concurrently with vips (cores 6-7), together fully occupying all spare cores 2-7 on node-a. The 8-core node is required to host both jobs without CPU contention.
- **canneal**: **node-b-4core**. As shown in Part 2a, canneal is nearly insensitive to all interference types due to its DRAM-bound random access pattern, but its memory-bandwidth footprint could still affect memcached if co-located on the same node. Placing it on node-b ensures that its access pressure does not reach memcached's working set, and the 4 cores available fully suffice given its sub-linear scaling profile from Part 2b.
- **freqmine**: **node-a-8core**. Based on the speedup data from Part 2b, freqmine scales effectively with more threads. We allocated 6 cores and with that 6 threads, utilizing all spare cores on node-a. This requires the 8-core node, as no other node provides 6 memcached-free cores.
- **radix**: **node-a-8core**. Radix was constrained to node-a since experiments placing it on node-b resulted in the job consistently crashing regardless of core assignment or Kubernetes resource configuration. After further research, we believe to attribute this behavior to the size of the native dataset of radix which exceeds available RAM node-b-4core has to offer since radix's permutation-heavy algorithm can burst DRAM consumption significantly which results in a forced placement on node-a-8core.
- **streamcluster**: **node-b-4core**. Like canneal, streamcluster is placed on the dedicated 4-core node to avoid interfering with memcached. As the longest running job, it was important to not let it compete for resources with memcached on node-a. Part 2b also shows sub-linear scaling for streamcluster at higher thread counts, meaning the 4 cores on node-b are sufficient for an efficient execution. Streamcluster runs sequentially after canneal, keeping node-b fully occupied until the end of the experiment.
- **vips**: **node-a-8core**. Vips is assigned 2 cores and is co-scheduled concurrently with blackscholes, together filling all spare cores 2-7 on node-a. The non-overlapping taskset ranges ensure zero CPU contention between the two jobs.

- Which jobs run concurrently / are colocated? Why?

Answer: Blackscholes and vips run concurrently on node-a-8core, pinned to non-overlapping core ranges (cores 2-5 and cores 6-7 respectively). This choice is driven by three factors: both jobs are short-running, their combined core usage exactly fills

all spare cores 2–7 on node-a, and their disjoint taskset assignments make CPU-level interference impossible by construction. Initial experiments confirmed that running them sequentially negatively affects the makespan. On node-b-4core, canneal and streamcluster run sequentially. Streamcluster starts as soon as canneal finishes, keeping node-b fully occupied throughout. Furthermore, all batch jobs on node-a are isolated from memcached by taskset, ensuring no CPU contention with cores 0–1 on which memcached is running.

- In which order did you run the 7 batch jobs? Why?

Order:

node-a-8core: freqmine, blackscholes + vips, barnes, radix

node-b-4core: canneal, streamcluster

Why: freqmine and canneal are launched first simultaneously (one per node) since they are one of the more resource-hungry and longest-running jobs. Thus starting them early is critical for minimizing the makespan. Once freqmine completes on node-a, blackscholes and vips are co-scheduled together to fully exploit all spare cores simultaneously. Streamcluster starts on node-b as soon as canneal finishes, maintaining full utilization of that node and runs until the end. Barnes follows on node-a after the blackscholes/vips wave and radix is launched last as the shortest job. This ordering with scheduling the longest job first ensures the critical path is as short as possible.

- How many threads have you used for each of the 7 batch jobs? Why?

Answer:

- **barnes:** 6 threads. Barnes is given all 6 spare cores on node-a. Part 2b shows it scales well with thread count, thus it makes sense to have full utilization of the available cores.
- **blackscholes:** 4 threads. Thread count matches the 4-core allocation. It runs concurrently with vips which occupies cores 6–7, so expanding blackscholes beyond 4 cores is not possible without introducing contention.
- **canneal:** 4 threads. Canneal fills all 4 cores of node-b. Although Part 2b shows its scaling is not linear, 4 threads still provide a significant runtime reduction over fewer threads.
- **freqmine:** 6 threads. Freqmine is pinned to all 6 spare cores on node-a and runs alone. Part 2b shows freqmine scales effectively with thread count, justifying full utilization of the available cores.
- **radix:** 8 threads. Empirical testing demonstrated that using more threads than available cores reduces runtime for radix, consistent with the observation in Part 2b that radix scales very well and that sub-linear users benefit from oversubscription due to the bursty nature of the workload. Furthermore, having thread sizes that are not a power of 2 have shown to never terminate.
- **streamcluster:** 4 threads. Streamcluster fills all 4 cores of node-b sequentially after canneal. 4 threads fully utilize the available node capacity.
- **vips:** 2 threads. Vips is co-scheduled concurrently with blackscholes, leaving only cores 6–7 available. The thread count is therefore constrained to 2 to match the remaining core allocation and avoid contention with blackscholes on

cores 2–5.

- Which files did you modify or add and in what way? Which Kubernetes features did you use?

Answer: We modified the YAML files in the parsec-benchmarks folder to:

- Run each job using taskset, specifying CPU core affinity for the PARSEC process to ensure isolation from memcached and from concurrently running jobs.
- Specify a nodeSelector policy, forcing each job to run on a specific node type (node-a-8core or node-b-4core).
- Specify resources.requests and resources.limits to control pod placement and prevent unintended co-scheduling by the default Kubernetes scheduler.

Kubernetes features used: nodeSelector, resources.requests, resources.limits.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer: In general, we observed that oversubscribing threads relative to cores does not significantly reduce runtime for most jobs, with the exception of radix, which benefits from 8 threads on 6 cores due to its synchronization-heavy phases that leave cores momentarily idle. This is somewhat counterintuitive since several PARSEC benchmarks demonstrated linear or near-linear scaling in Part 2b. However, the scaling curves from Part 2b were measured in isolation, while in our setup cores are already fully allocated and the benefit of additional threads is outweighed by the overhead of context-switching. For node assignments, we experimented with placing memcached on node-b-4core instead of node-a-8core. This proved worse for two reasons. First, memcached running on node-b shared the node with canneal and streamcluster, both of which are memory-bandwidth-intensive and caused measurable interference on memcached’s p95 latency. Second, the freed cores on node-a did not translate into a shorter makespan, as the bottleneck remained the sequential execution of barnes, radix and streamcluster rather than the number of cores available to earlier jobs. We also experimented with a second scheduling policy that assigned barnes and radix to only cores 2–5, leaving cores 6–7 idle during those waves. This simpler policy achieved a comparable makespan but wasted available resources with cores 6–7 sitting idle during the barnes and radix phases. The final policy addresses this by expanding both jobs to cores 2-7, giving barnes 6 threads and radix 8 threads, which reduces their individual runtimes and brings the total makespan down by several seconds across all three runs.

2. [34 points] Describe and analyze the AI-generated policy.

(a) [17 points] **Report the performance:**

- The AI-evolved policy maintains the same DAG topology as the Part 3.1 hand-crafted baseline but introduces mild thread oversubscription on node-a: blackscholes runs with 5 threads on 4 cores (compared to the baseline of 4) and barnes with 7 threads on 6 cores (compared to the baseline of 6). The node-b chain and all core assignments remain unchanged.

job name	mean time [s] (Baseline)	std [s]	mean time [s] (AI)	std [s]
barnes	35.3	1.2	33.7	1.5
blackscholes	41.3	1.5	51.0	10.8
canneal	75.0	1.0	74.7	0.6
freqmine	92.7	3.1	98.7	12.5
radix	11.0	1.7	10.0	1.0
streamcluster	141.0	1.0	139.3	1.5
vips	52.0	1.7	55.3	9.3
total time (makespan)	219.0	2.0	225.3	2.1

Table 3: Mean execution time and standard deviation per batch job across 3 runs for the Part 3.1 hand-crafted baseline and the AI-evolved policy (Part 3.2). Baseline values are reproduced from Part 3.1 for direct comparison. The AI policy achieves a mean makespan of 225.3s (2.9% slower than the 219.0s baseline).

The mean SLO violation ratio across the three AI policy runs is **0.017**: Runs 1 and 2 experienced zero violations, while Run 3 recorded 1 violation out of 20 samples (ratio = 0.05), where the p95 latency spiked to 1213 μ s. This violation occurred during the concurrent blackscholes/vips phase, which corresponds to the oversubscription bottleneck identified in the analysis below. While a sample size of $n = 3$ runs is insufficient to fully isolate policy-induced variance from background cluster noise, the timing of the violation aligns with the oversubscription on cores 2–5. The additional thread likely increases memory bandwidth pressure, which can contribute to occasional memcached p95 spikes.

The AI policy also exhibits higher variance than the baseline, particularly for blackscholes ($\sigma = 10.8$ s vs. 1.5s) and freqmine ($\sigma = 12.5$ s vs. 3.1s). This behavior is a known trade-off of thread oversubscription: forcing 5 threads to contend for 4 cores increases the execution time’s sensitivity to OS scheduling decisions and cache-state variability.

- Figures 6–8 show the memcached p95 latency over time for each of the three AI policy runs, with annotations indicating when each batch job started and ended and on which node it executed. The schedule structure is visible: freqmine and canneal launch concurrently at $t = 0$; after canneal completes ($t \approx 75$ s), streamcluster starts on node-b; after freqmine completes ($t \approx 90$ –110s), blackscholes and vips run in parallel on node-a; then barnes and finally radix. Node-b’s streamcluster is consistently the last job to finish, determining the makespan at $t \approx 220$ –227s.

Answer: See Figures 6–8.

- Figure 9 shows two line plots tracking policy performance across 30 OpenEvolve iterations on the real cluster. The left plot shows worst p95 latency per iteration, with the 1000 μ s SLO threshold marked. The right plot shows SLO violation ratio. Iterations 17 and 23 are marked with $\times$ symbols at -1.0 as they failed due to infrastructure timeouts (SSH to the load-generator VM hung), not due to invalid schedules. The best combined score of 1.0081 was achieved at iteration 20.

Answer: See Figure 9.

(b) [17 points] Analysis of the Evolutionary Process:

Answer:

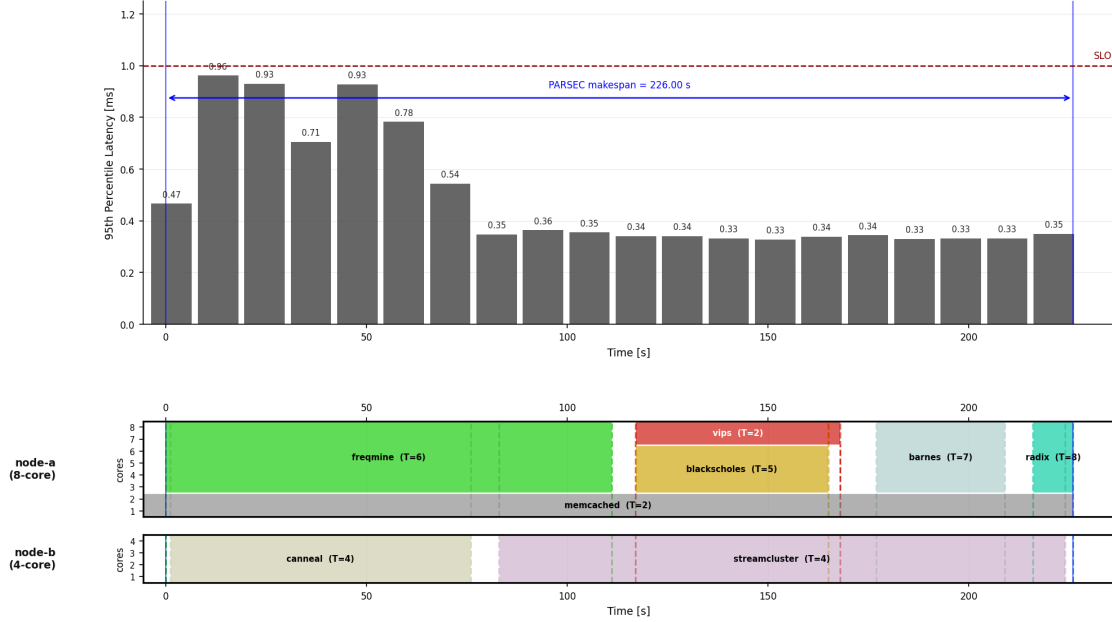


Figure 6: AI policy Run 1: p95 latency over time. Makespan: 226 s. 0 SLO violations. Worst p95: 961 μ s.

We designed two strategies for using Openevolve in this project. One was an offline simulation, the other a direct deployment on the cluster with real measurements as feedback. **Strategy 1 (simulator-based)** utilized the Part-2 discrete-event interference simulator as a fast proxy evaluator (~ 1 s per candidate). We performed 80 iterations with **Llama-3.3-70B-Instruct** and 62 iterations with **Qwen-2.5-72B-Instruct**, both hosted via the SwissAI API. The MAP-Elites population for this simulator strategy (200 programs, 4 islands) was configured to facilitate structural exploration. The 4 islands allowed for the parallel migration of distinct policy families (e.g., node-b oversubscription vs. dependency relaxation) to mitigate premature convergence, while the 200-program archive provided sufficient capacity to retain diverse Pareto-optimal candidates across the makespan/SLO trade-off. We seeded this population with 6 hand-crafted policy variants: (i) 8-thread oversubscription on node-b for canneal and streamcluster, (ii) barnes depending only on blackscholes (dropping the vips dependency), (iii) radix on cores 6–7 running in parallel with barnes, (iv) symmetric 3t/3t split for blackscholes/vips, (v) fregmine reduced to 4 cores with vips starting at $t = 0$, and (vi) a combination of (ii) and (iii). These seeds provided a diverse set of structural starting points for the population from the outset.

Strategy 2 (real-cluster) deployed each candidate on the live Kubernetes cluster and measured latency with mcperf, requiring ~ 5 minutes per iteration. We ran 30 iterations with Llama-3.3-70B-Instruct over ~ 2.5 hours, using a smaller population (50 programs, 2 islands) and a higher exploitation ratio (0.7 vs. 0.6 default). The configuration of 50 programs and 2 islands was determined by time constraints: given the ~ 5 min/iteration runtime, a 30-iteration run populates only $\sim 60\%$ of the archive. A smaller archive size helps ensure each slot receives sufficient samples to guide selection,

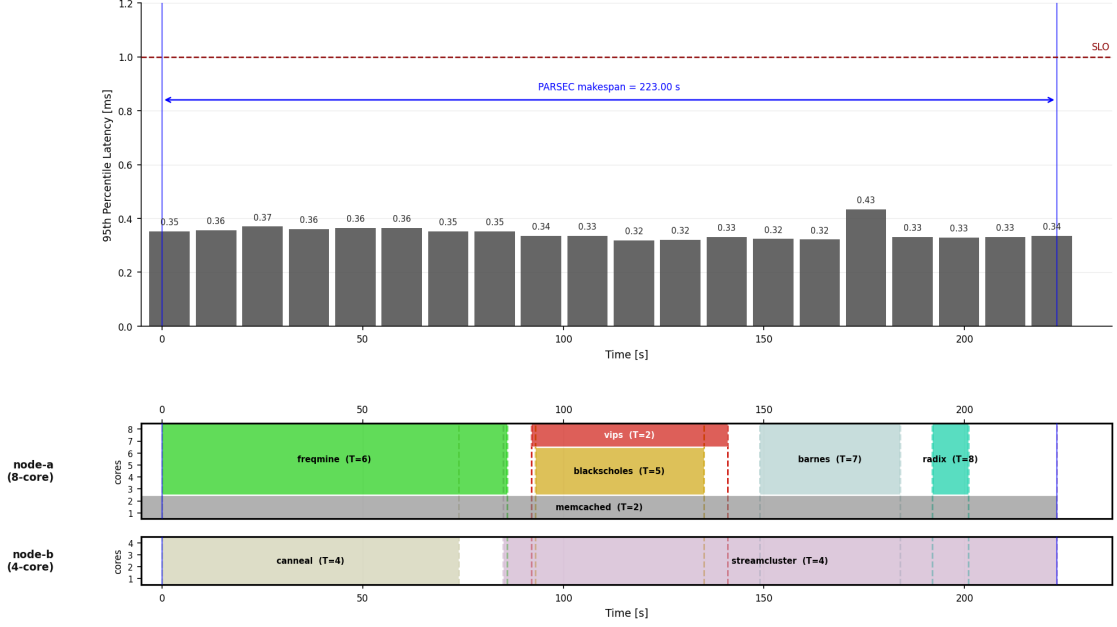


Figure 7: AI policy Run 2: p95 latency over time. Makespan: 223 s. 0 SLO violations. Worst p95: 433 μ s.

and 2 islands represents the minimum configuration required to maintain migration-based diversity without over-allocating the limited sample budget. The exploitation ratio was increased to 0.7 (relative to the 0.6 default) to favor known-good policies, as random structural mutations under noisy real-cluster feedback are less likely to produce consistent improvements. The final submitted policy was derived from Strategy 2

We did not use a transfer pipeline (pre-filter with simulator, validate on cluster) because early experiments showed that the simulator’s results were significantly worse than our baseline in 3.1.

Success metric design. The combined score balances makespan minimization against SLO compliance using a two-tier penalty:

$$\text{combined} = \underbrace{\frac{219.0}{\text{makespan.s}}}_{\text{speedup}} - \underbrace{\frac{\max(0, \overline{p_{95}} - 800)}{200}}_{\text{soft penalty}} - \underbrace{5.0 \times r_{\text{SLO}}}_{\text{hard penalty}} \quad (2)$$

where 219.0s is our measured Part 3.1 baseline makespan, $\overline{p_{95}}$ is the mean p95 latency in microseconds, and r_{SLO} is the fraction of mcperv samples exceeding the 1000 μ s SLO. A score of 1.0 means matching the baseline with zero SLO impact; > 1.0 means faster. The *soft penalty* creates a continuous gradient above 800 μ s (200 μ s below the SLO), discouraging latency creep before it crosses the threshold. The *hard penalty* uses a $5\times$ multiplier, ensuring that even a 10% violation ratio incurs -0.5 , which overwhelms any realistic speedup. This asymmetry steers the LLM strongly away from SLO-violating schedules.

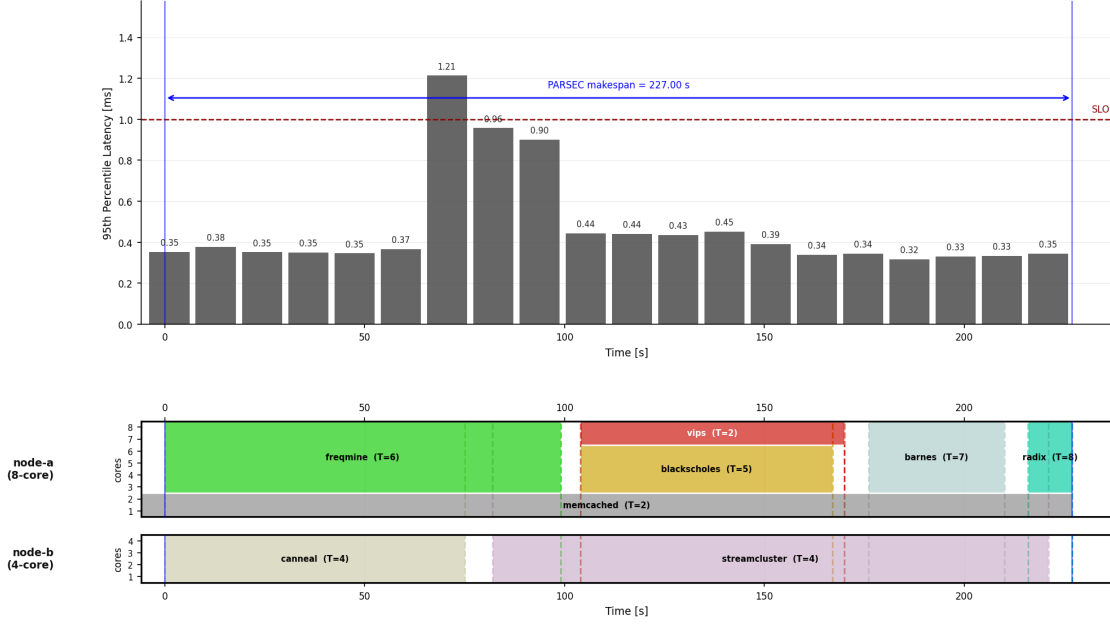


Figure 8: AI policy Run 3: p95 latency over time. Makespan: 227s. 1 SLO violation at $t \approx 170$ s (p95 = 1213 μ s) during the blackscholes/vips phase.

Initial policy and rationale. We seeded both strategies with the hand-crafted Part 3.1 baseline:

```
node-a: freqmine(6t, c2-7) → [blackscholes(4t, c2-5) || vips(2t, c6-7)] →
      barnes(6t, c2-7) → radix(8t, c2-7)
node-b: canneal(4t, c0-3) → streamcluster(4t, c0-3)
```

We chose this because: (1) it is a *known-valid* schedule with measured performance (219s makespan, zero SLO violations), providing the LLM a working solution rather than requiring it to search the exponential space from scratch; (2) it encodes Part-2 domain knowledge—high-interference jobs (canneal: +700 μ s p95, streamcluster: +850 μ s p95) are on node-b away from memcached, and dependency chains prevent concurrent interference on shared cores; (3) it enables *incremental* optimization, where the LLM can tweak individual parameters while maintaining schedule validity.

Policy evolution: what the LLM discovered. *Strategy 1 (simulator).* The LLM’s most interesting result was the **node-b oversubscription exploit**: running canneal and streamcluster with 8 threads on 4 cores ($2\times$ oversubscription). The simulator’s model predicts only an 11% penalty per extra thread ($\text{slowdown} = (t/c)^{0.15}$), yielding a large net speedup. This reduced the simulated node-b chain from ~ 230 s to ~ 156 s. The best simulated score reached **1.21** (190s makespan) after 8 seeded iterations. The Qwen model independently found a complementary structural change: relaxing barnes’s dependency to only blackscholes (dropping the vips wait), allowing earlier start of the node-a tail.

Strategy 2 (real cluster). On the cluster, the LLM evolved much more conservatively,

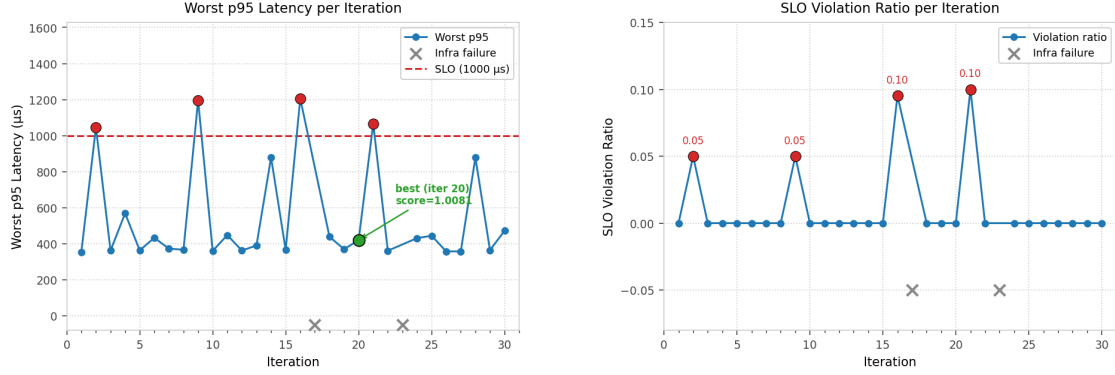


Figure 9: Evolution trajectory across 30 real-cluster iterations. **Left:** worst p_{95} latency per iteration with the 1000 μs SLO threshold (dashed red). **Right:** SLO violation ratio. Iterations 17 and 23 (grey $\times$) failed due to infrastructure timeouts (SSH hangs), not invalid schedules. The best combined score of 1.0081 was achieved at iteration 20 (green dot).

making only thread-count adjustments while preserving the DAG topology. Table 4 shows the five noteworthy policy changes:

Iter	Score	Makespan [s]	Change from previous best
0	0.959	228.3	Initial seed (bs=4t, barnes=6t)
1	0.970	225.8	Minor thread rebalancing
3	0.980	223.6	blackscholes $\rightarrow$ 5t
5	0.980	223.4	barnes $\rightarrow$ 7t
12	0.992	220.7	Tighter scheduling (same params)
20	1.008	217.2	Favorable cluster conditions

Table 4: Evolution milestones on the real cluster (30 iterations). The LLM made only incremental changes to thread counts; the DAG topology and core assignments remained identical to the baseline throughout.

The conservative evolution is not a design choice we imposed: the LLM received noisy feedback (the same schedule scored 0.98 in one iteration and 0.43 in another due to transient interference), making it difficult to distinguish genuine improvements from measurement variance. The LLM could not confidently identify structural improvements (different DAGs, different node assignments) because of the heavy noise. This is a fundamental trade-off: the simulator enables bold structural exploration (at the cost of modeling fidelity), while the real cluster provides ground truth (at the cost of noisy measurements).

Note on the iteration-20 vs. validation gap. During the search, iteration 20 achieved a 217.2s makespan, but subsequent validation runs averaged 225.3s. This discrepancy might stem from high environmental variance: across 30 iterations, the *same* schedule produced scores ranging from 0.43 to 0.98, with runtime fluctuations up to $\sim 20\%$ caused by GCP hardware heterogeneity. Thus, iteration 20 either encountered exceptionally favorable hardware—making 217.2s an outlier (consistent with Table 4)—or the validation runs faced adverse hardware that biased the mean upward. Either case reinforces the observation that the LLM operated on a noisy signal, highlighting that

real-cluster evolution is fundamentally bottlenecked by measurement variance.

Why the AI policy performed worse than the baseline. The AI policy achieves a mean makespan of 225.3s versus the baseline’s 219.0s (2.9% slower), with a mean SLO violation ratio of 0.017 (versus 0.000). The root cause is that **blackscholes oversubscription backfired**: running 5 threads on 4 cores increased its mean execution time from 41.3s to 51.0s (+23%), with standard deviation rising from 1.5s to 10.8s. The extra thread contends for the L2 cache and memory bandwidth on the shared cores 2–5. Since blackscholes and vips run concurrently after freqmine, this delay pushes back the start of barnes and radix, extending the node-a critical path by ~6s.

This reveals a **sim-to-real gap** in the oversubscription model. The simulator’s best policy (2× oversubscription on node-b) predicted a 1.21× speedup; on the real cluster, even the mild 1.25× oversubscription (5t on 4 cores) caused a net slowdown. The simulator is valuable for rapidly filtering *invalid* strategies (wrong nodes, missing jobs, dependency cycles ie. it caught 970 such errors in our largest run), but it cannot reliably rank *close alternatives* where the difference is a single thread.

Hallucination identified during the process. The clearest hallucination was the LLM **inventing non-existent CPU cores**. In 970 out of 6,966 programs (13.9%) generated during the simulator run (`seeded_run01`), the LLM assigned jobs to cores that do not physically exist. Node-a has 8 vCPUs with usable PARSEC cores [2–7] (cores 0–1 are reserved for memcached); node-b has 4 vCPUs with cores [0–3]. Despite this being stated explicitly in the system prompt, the LLM repeatedly generated assignments such as:

```
Action("blackscholes", "node-a", (2,3,4,5,6,7,8,9), 8, ...)
Action("canneal",      "node-b", (0,1,2,3,4,5,6,7), 8)
```

hallucinating cores 8–9 on node-a and cores 4–7 on node-b. One program even assigned cores [4,5,6,7,8,9,10,11] to a node-b job—12 cores on a 4-core machine. The LLM apparently generalized from the stated vCPU count without respecting the enumerated core lists provided in the prompt. These errors occurred across 31 distinct iterations and were caught by the evaluator’s core-range validator (scoring −1.0), representing 76.9% of all failures in that run.

A subtler hallucination involved the LLM generating **semantically meaningless optimization code**. In the failed cluster run, the LLM produced a 95-line “prioritization” algorithm that attempted to sort the Action list through three passes, believing that list position determines execution order. In fact, execution order is determined solely by the `start_after` DAG—list position is irrelevant. The code also contained a type confusion, passing string job names to `list.index()` on a list of Action objects. This pattern persisted across 9 consecutive iterations (checkpoints 26–30) as the LLM kept evolving variants of the same flawed structure.

Finally, the simulator itself induced a form of hallucination: the LLM’s best simulator policy schedules blackscholes and vips on the *exact same four cores* concurrently (both on cores 2–5 after freqmine). The simulator penalizes this overlap by only 11%, predicting a net speedup from parallelism. On real hardware, two compute-intensive jobs sharing L2 caches would cause 50–100% slowdown from cache thrashing. The LLM *correctly*

optimized for a flawed model, producing a policy that exploits the simulator rather than reflecting physical reality.

The persistence of the core-invention errors across 31 iterations suggests that OpenEvolve’s prompt-conditioned mutation is insufficient to enforce hard structural constraints on its own; a constrained decoding step or a post-hoc repair pass would likely reclaim the 13.9% of wasted samples and improve overall sample efficiency.

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.

Part 4 [75 points]

1. [19 points] Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 2 \
    --scan 5000:125000:10000
```

a) [8 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. achieved QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

- Memcached with $T=1$ thread, $C=1$ core
- Memcached with $T=1$ thread, $C=2$ cores
- Memcached with $T=1$ thread, $C=3$ cores
- Memcached with $T=2$ threads, $C=1$ core
- Memcached with $T=2$ threads, $C=2$ cores
- Memcached with $T=2$ threads, $C=3$ cores
- Memcached with $T=3$ thread, $C=1$ cores
- Memcached with $T=3$ thread, $C=2$ cores
- Memcached with $T=3$ threads, $C=3$ cores

Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

Plots: The plot is provided in Figure 10, which is averaged across 3 runs with ± 1 standard deviation error bars in both dimensions marked by the x-axis for QPS and y-axis for p95 latency.

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores and how this relates to the target QPS.

Summary: Memcached throughput scales with the number of threads, but only when enough cores are available to run them. With $T=1$ thread, performance is bounded by a single core regardless of C : all three $T=1$ configurations saturate at approximately 65K-70K QPS and violate the 0.8 ms SLO above that, because the single thread cannot be distributed across additional cores. Adding cores without increasing threads therefore yields no benefit. With $T=2$ or $T=3$ threads, the number of cores becomes the determining factor: $T=2$ with $C=1$ also saturates near 70K QPS where only one core is available to service both threads, whereas $T=2$ or $T=3$ paired with $C \geq 2$ cores sustain significantly higher throughput. The best configuration is $T=3$ threads on $C=3$ cores: maintains p95 latency below 0.6 ms across the entire 5K-125K QPS sweep, demonstrating that throughput scales with the effective parallelism $\min(T, C)$.

b) [2 points] To support the highest load in the trace (around 125K QPS) without violating the 0.8ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

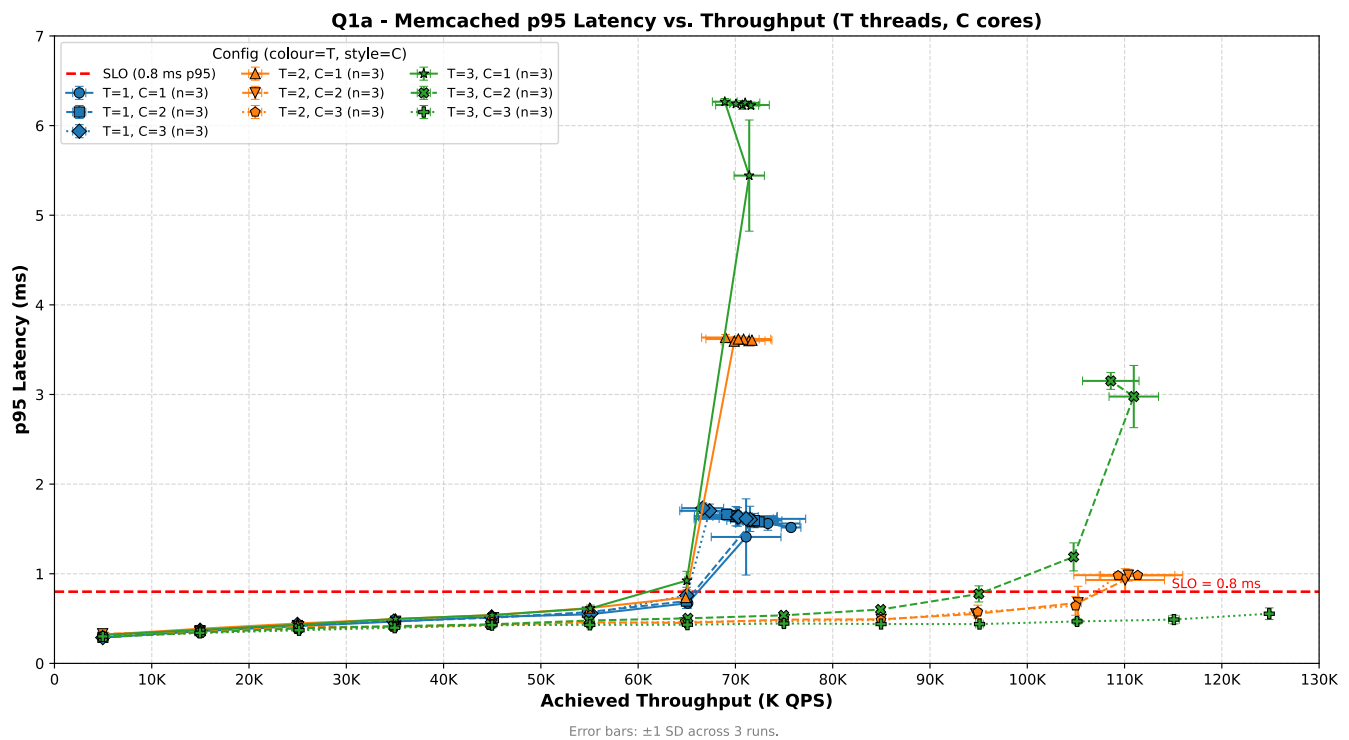


Figure 10: Comparison of interferences via 95th percentile vs achieved QPS. Points represent means across runs. Error bars show standard deviation.

Answer: $T = 3$ threads, $C = 3$ cores.

From the data, $T=3$, $C=3$ is the only configuration that sustains the full 125K QPS sweep within the SLO with a mean $p95 = 0.554$ ms at 125K QPS. All other configurations either saturate earlier or only narrowly miss at high QPS. E.g. for early saturation: $T=1$, $T=2/C=1$ and $T=3/C=1$ caps at $\approx 70K$, while $T=3/C=2$ violates SLO at 105K. E.g. narrow target-miss configurations are: $T=2/C=2$, $T=2/C=3$ both exceeding 0.8 ms SLO at 110K, while $T=3/C=2$ exceeds SLO at $\approx 100K$ QPS. The latter is explained by considering that 3 threads fighting for 1 single core lead to severe scheduling overhead, which also explains severe latency in the early saturation phenomena for the $T=3/C=1$ configuration.

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you propose to use to guarantee the 0.8ms 95th percentile latency SLO while the load varies between 5K to 125K QPS?

Answer: $T = 3$ threads.

The number of threads must be fixed at launch but cores can be adjusted dynamically. Thus, the constraint is that T must be sufficient to saturate up to 3 cores at peak load of 125K QPS, since a single thread can only run on one core at a time without massive overhead, implying that adding cores beyond T brings no benefit.

From the Q1 a) results, we get that $T=1$ peaks at $\approx 65K$ QPS regardless of C , since the single thread is the bottleneck: it cannot meet the SLO above 65K even with $C=3$. For $T=2$, together with $C=2$ or $C=3$ can sustain up to $\approx 105K$ QPS within SLO, but violates it at 110K target with $p95 \approx 0.93$ – 0.98 ms, hence insufficient for 125K QPS of peak load. Only $T=3/C=3$ can sustain the full 5K–125K sweep within SLO, with max $p95 = 0.554$ ms at 125K QPS. We therefore propose $T=3$, which ensures that all 3 cores can be fully utilized by memcached threads when needed, without over-provisioning threads beyond the number of available cores.

d) [8 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$, $C = 2$ and $C = 3$. Use the aforementioned `mcperrf` command to sweep QPS from 5K to 125K.

Measure the CPU utilization on the memcached server at each load time step. Plot the performance of memcached using 1-core ($C = 1$), 2 cores ($C = 2$) and 3 ($C = 3$) in **three separate graphs**, for $C = 1$, $C = 2$ and $C = 3$, respectively. In each graph, plot achieved QPS on the x-axis, ranging from 0 to 125K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 0.8ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$, 200% for $C = 2$ and 300% for $C = 3$) on the right y-axis. For simplicity, we do not require error bars for these plots.

Plots: The plots are provided in Figures 11, 12, 13 in separate graphs based on the core count with the requested format of axis and measurements.

2. [17 points] You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 110K in 15 second time intervals. Use the following command to run this trace:

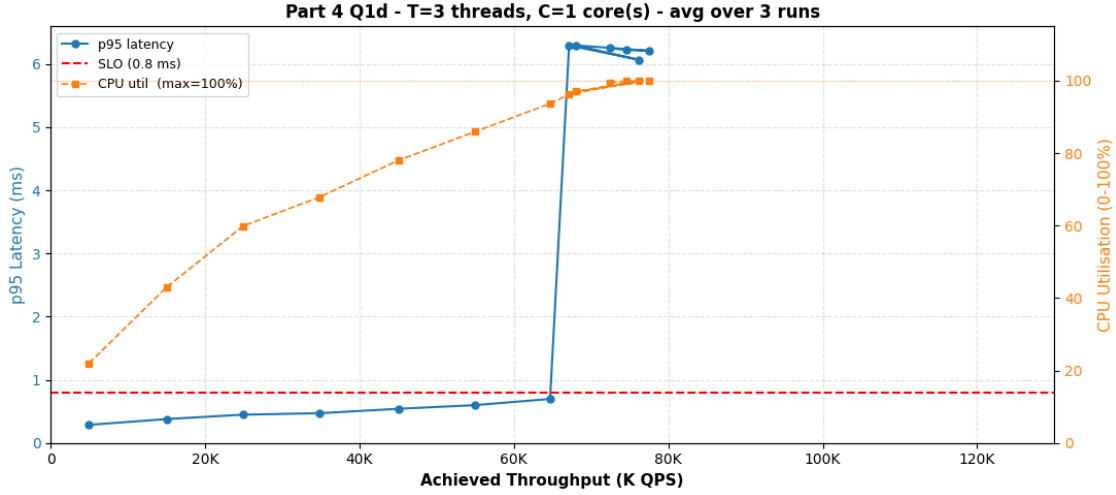


Figure 11: Memcached p95 latency and CPU utilization vs. achieved throughput for T=3 threads, C=1 core. With a single core, memcached saturates at $\approx 65K$ QPS and violates the 0.8 ms SLO beyond that point, as the remaining two threads cannot be scheduled.

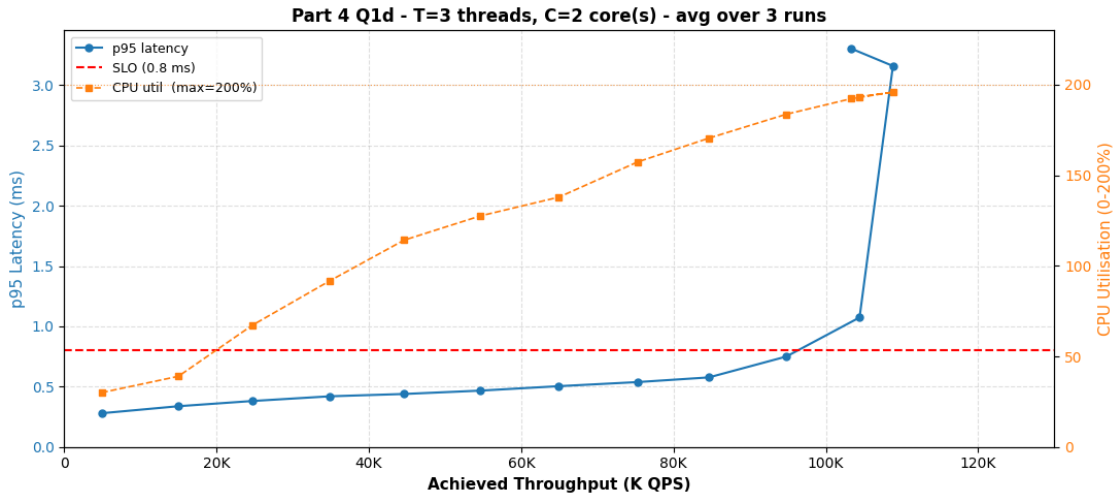


Figure 12: Memcached p95 latency and CPU utilization vs. achieved throughput for T=3 threads, C=2 cores. With two cores, memcached holds the SLO up to $\approx 95K$ QPS. Beyond that, the two cores become saturated and latency rises sharply.

```
$ ./mcpref -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcpref -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
  --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
  --qps_interval 15 --qps_min 5000 --qps_max 110000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.

For this and the next questions, feel free to reduce the mcpref measurement duration (`-t`

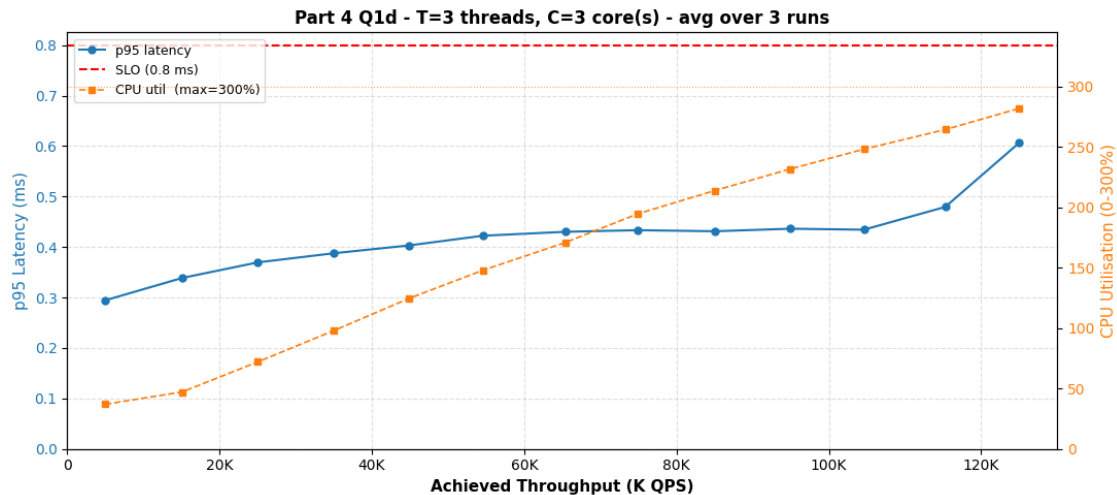


Figure 13: Memcached p95 latency and CPU utilization vs. achieved throughput for $T=3$ threads, $C=3$ cores. With three cores, memcached sustains the 0.8 ms SLO across the full 5K–125K QPS range, with peak p95 latency of 0.6 ms at 125K QPS.

parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the benchmarks (batch jobs) on the 4-core VM. The goal of your scheduling policy is to successfully complete all batch jobs as soon as possible without violating the 0.8ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The batch jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the batch jobs complete successfully and do not crash. Note that batch jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):

Answer: The controller is a Python process running on the memcache-server VM together with memcached and the batch jobs. The default policy mode is `stress_adaptive`. At startup, it pins memcached to cores 0, 1, and 2 and does not start batch jobs while it classifies the stress situation from early memcached total CPU changes. If changes are slow, as in the 15 s trace, it switches to `core_fast` mode. If changes are fast or ambiguous, it switches to `bounded_total_gate_slotb_plus` mode. At start (min 15 s of runtime), the classifier selected `core_fast` mode with median change spacing around 15.02 s. `core_fast` mode gives memcached a variable prefix of cores and runs one batch job on every currently free core. When memcached expands, only the batch job on the stolen core is paused. When memcached shrinks, that core is immediately returned to batch work.

This gives high parallelism while still preserving the 0.8 ms p95 SLO.

- How do you decide how many cores to dynamically assign to memcached? Why?

Answer: The controller does not receive the trace interval as an input. It first infers the stress situation by sampling total memcached CPU every 0.25 s. If the median spacing between large CPU changes is at most 10.5 s, the run is classified as stress situation and the safer policy mode is active. If the spacing is at least 12.5 s, the run is classified as relaxed and core_fast mode is active. In core_fast mode, memcached receives 1, 2, or 3 cores according to total CPU utilization. It grows from 1 to 2 cores when total CPU exceeds 80 percent. It grows from 2 to 3 cores when total CPU exceeds 160 percent. It shrinks from 2 to 1 core when total CPU falls below 95 percent. It shrinks from 3 to 2 cores when total CPU falls below 170 percent, or directly to 1 core when total CPU falls below 125 percent. Total CPU is used instead of per core CPU because it is stable across different memcached core allocations.

- How do you decide how many cores to assign each batch job? Why?

Answer: In the relaxed case, the policy uses one batch job per free core. Core 0 is always available to memcached. Cores 1 and 2 move between memcached and batch work depending on memcached total CPU. Core 3 is always reserved for batch work. When memcached has 1 core, batch jobs may use cores 1, 2, and 3. When memcached has 2 cores, batch jobs may use cores 2 and 3. When memcached has 3 cores, only core 3 is available for batch work. This design avoids running one large batch job with too many software threads on too few physical cores. It also prevents a memcached expansion from stopping all batch progress, since only the job on the stolen core is paused.

- `barnes`: 1 core.
- `blackscholes`: 1 core.
- `canneal`: 1core.
- `freqmine`: 1 core.
- `radix`: 1 core.
- `streamcluster`: the minimum is 1 cores while maximum reaches 3 cores. The long makespan of streamcluster made it the desirable candidate for a job running potentially in multiple cores.
- `vips`: 1 core.

- How many threads do you use for each of the batch job? Why?

Answer: In core_fast mode, almost all batch jobs use one thread. This is deliberate because the policy modality directly maps one batch job to one physical core. The exception is streamcluster, which uses 3 threads because it is the final long job and can expand to extra free cores after the one core jobs finish. This choice follows as broad concurrency is more valuable here than giving many threads to a single job while other cores are idle.

- `barnes`: 1 thread.
- `blackscholes`: 1 thread.
- `canneal`: 1 thread. Memory latency bound, extra threads do not help.
- `freqmine`: 1 thread.

- **radix**: 1 thread. Memory bandwidth bound, extra threads do not help.
 - **streamcluster**: 3 threads.
 - **vips**: 1 thread.
- Which jobs run concurrently / are collocated and on which cores? Why?
Answer: In the first 15 s run, the adaptive controller usually selects `core_fast` modality if not under stress. The first jobs started were `freqmine` on core 3, `canneal` on core 1, and `barnes` on core 2. After that, whenever a one core job finished, the next job was started on the freed core. The observed order was `freqmine`, `canneal`, `barnes`, `vips`, `blackscholes`, `radix`, and `streamcluster`. During high `memcached` load, `memcached` used cores 0, 1, and 2, so the jobs on cores 1 or 2 were paused. During lower load, those jobs were resumed. `Streamcluster` was started last with 3 threads and was expanded to cores 1, 2, and 3 whenever `memcached` released them. This keeps the machine busy while making `memcached` preemption local to the exact core it needs. All of the mentioned mechanisms can be empirically verified by looking at Figures 14, 15, 16.
 - In which order did you run the batch jobs? Why?
Order: `freqmine`, `canneal`, `barnes`, `vips`, `blackscholes`, `radix`, `streamcluster`.
Why: The order is chosen for the `core_fast` mode to lean the policy for fast performance, where most jobs run as one core jobs. It starts several independent one thread jobs early so that all free cores are occupied. `Streamcluster` is placed last because it is the only job with 3 threads and can use multiple cores after the smaller one core jobs have completed. This makes the tail shorter because `streamcluster` can absorb any cores released by `memcached` near the end. In the three official qps-interval 15 s runs, this produced makespans of about 838.39 s, 829.84 s, and 843.43 s, all below 900 s.
 - How does your policy differ from the policy in Part 3? Why?
Answer: In Part 3, `memcached` load was constant at 30K QPS and the scheduling problem was mostly static. In Part 4, the load varies between 5K and 110K QPS on a single 4 core VM, so the number of cores needed by `memcached` changes during the run. The Part 4 controller therefore updates `memcached` affinity with taskset while jobs are running. It also updates Docker cpusets and pauses or unpauses batch containers when `memcached` takes or releases cores. The adaptive classifier is also new. It decides at runtime whether the run is relaxed enough for the fast policy or whether the safer policy is required. So it infers the stress of the infrastructure on the fly.
 - How did you implement your policy? e.g., docker cpu-set updates, taskset updates for `memcached`, pausing/unpausing containers, etc.
Answer: The entry point is `scripts/controller.py`. It first runs `stress_adaptive` and then replaces itself with the selected concrete policy. This is done with `os.execvpe`, so the same controller process restarts with `CONTROLLER_POLICY` set to either `core_fast` or `bounded_total_gate_slotb_plus`. `Memcached` is pinned with taskset because Docker cpuset does not reliably constrain the native `memcached` process. Batch jobs are Docker containers, and their core allocation is controlled with `container.update(cpuset_cpus=...)`. When `memcached` expands, the controller pauses the job on the core being taken. When `memcached` releases a core, the controller unpauses the job or starts a new one there.

SchedulerLogger records all required start, update.cores, pause, unpause, and end events.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer: The main design choice was to separate two cases. For relaxed traces, the goal is minimum makespan with no SLO violations. For stress situation traces, the goal is SLO protection first, even if the makespan is longer. The classifier makes this choice from early memcached total CPU changes, not from an explicit trace parameter, which would not be allowed by the project statement.

Within the policy, the core.fast mode instead of running one large batch job at a time, it runs one job per free core. This increases parallelism and makes preemption cheap, since memcached only pauses the job on the exact core it needs. The tradeoff is that this policy is only safe when load changes are slow enough. On fast changing traces, the controller switches to bounded_total_gate.slotb.plus mode, which is slower but measured below the 3 percent SLO violation target.

The main comparison is against the previous legacy controller. The legacy controller used a bounded two slot design with per core utilization thresholds and longer hysteresis. It protected the SLO, but it often behaved almost sequentially and had much longer makespan. The latest adaptive controller reached 0.00 percent SLO violations in the qps-interval 15 s runs and completed all jobs in about 830 s to 843 s. This is faster because it keeps all free cores doing useful batch work and because streamcluster can expand at the end when cores become available.

3. [23 points] Run the following mcperf memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
  --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
  --qps_interval 15 --qps_min 5000 --qps_max 110000 \
  --qps_seed 2345
```

Measure memcached and batch job performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached. For each batch application, compute the mean and standard deviation of the execution time ² across three runs. Compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

²Here, you should only consider the runtime, excluding time spans during which the container is paused.

job name	mean time [s]	std [s]
barnes	167.1	1.0
blackscholes	109.3	2.0
canneal	264.0	4.2
freqmine	562.0	38.7
radix	40.7	0.9
streamcluster	327.3	18.8
vips	89.8	1.0
total time	837.2	6.9

Answer: The execution time of each batch job is computed as active runtime only, excluding all intervals where the corresponding container was paused. The total time is the makespan, i.e. the wall clock time from the first batch container start to the last batch container end. Across the three runs, the makespan was 838.4 s, 829.8 s, and 843.4 s, giving a mean of 837.2 s and a standard deviation of 6.9 s. The SLO violation ratio during the batch execution window was $0/56 = 0.0\%$ for Run 1, $0/56 = 0.0\%$ for Run 2, and $0/56 = 0.0\%$ for Run 3. Thus, memcached stayed below the 0.8 ms p95 latency SLO during all measured batch execution windows.

Include three plots – one for each of the three runs – with the following information: The x-axis should be the time from execution start to end. Use three subplots that share the same x-axis. The first should clearly indicate for each core what job it is running at all time (e.g. using colored boxes). Use the colors proposed in this template (you can find them in `main.text`). If you pause/unpause jobs as part of your scheduling policy this should also be clearly visible. The second subplot should include two y-axis where the left one shows the current QPS and the right one should include the achieved p95 latency. Finally, the third subplot should plot the cpu utilization of each core separately. The readability of your plot will be part of awarded points.

Plots: The plots are given in Figures 14, 15, 16. Notice that the graph shows every core hitting the 100% top of the scale during batch-intensive intervals.

4. [16 points] Repeat Part 4 Question 3 with a modified `mcperf` dynamic load trace with a 5 second time interval (`qps_interval`) instead of 15 second time interval. Use the following command:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
  --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
  --qps_interval 5 --qps_min 5000 --qps_max 110000 \
  --qps_seed 2345
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.

Part 4 - Q3 Run 1 with interval=15s

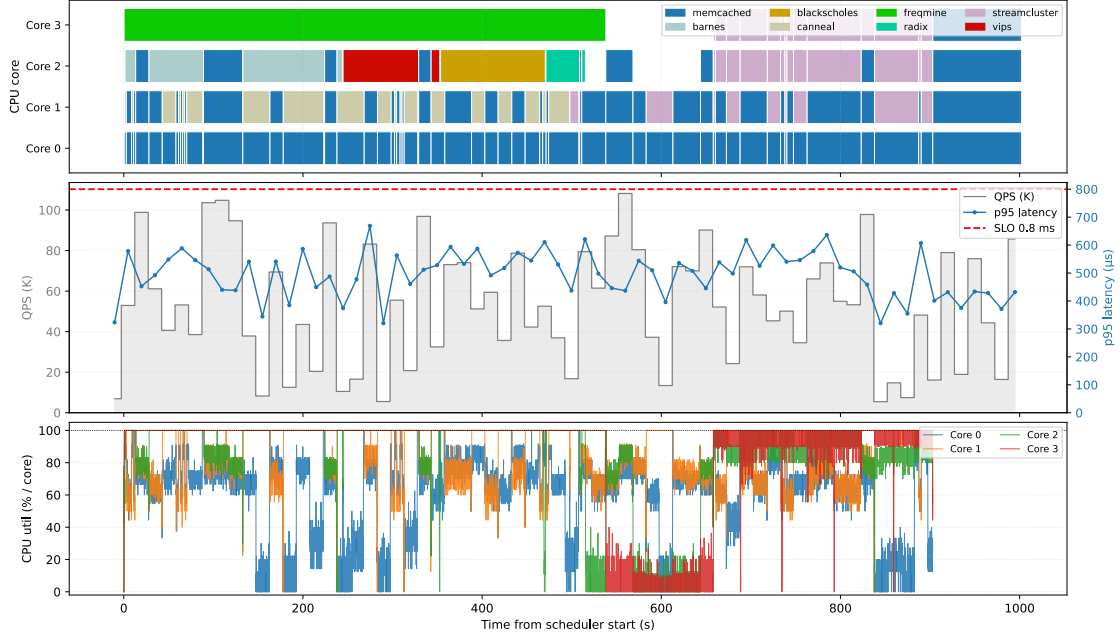


Figure 14: Part 4 Q3 Run 1, adaptive scheduler execution with interval=15 s. The controller achieves 0/56 SLO violations during the batch execution window, and 0/68 over the whole measurement trace. During the batch window, max p95 = 700.8 μ s and mean p95 = 528.7 μ s, remaining below the 0.8 ms threshold. The batch makespan is 838.4 s, and peak CPU utilization reaches 100% on all four cores during the heaviest phases, showing that the scheduler keeps the machine saturated while preserving memcached latency within the SLO.

Summary: With the 5 second interval trace, the policy’s adaptive classifier detects a stress situation and selects the safer `bounded_total_gate_slotb_plus` mode instead of the faster `core_fast` mode used in Question 3. The policy remains below the 3% SLO violation threshold, with 5 violations over 313 measurement points during the batch execution window, giving 1.60%. Compared to Question 3, the smaller interval increases load variability and makes the controller more conservative, so the makespan grows from about 837.2 s in Q3 to 1568.4 s in the 5 second run.

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints) with the 5-second time interval trace? The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

Answer: The SLO violation ratio is computed during the batch execution window, from the first batch container start to the last batch container end. For the 5 second interval trace, all seven jobs completed normally. There were 5 intervals with p95 latency above 0.8 ms out of

Part 4 - Q3 Run 2 with interval=15s

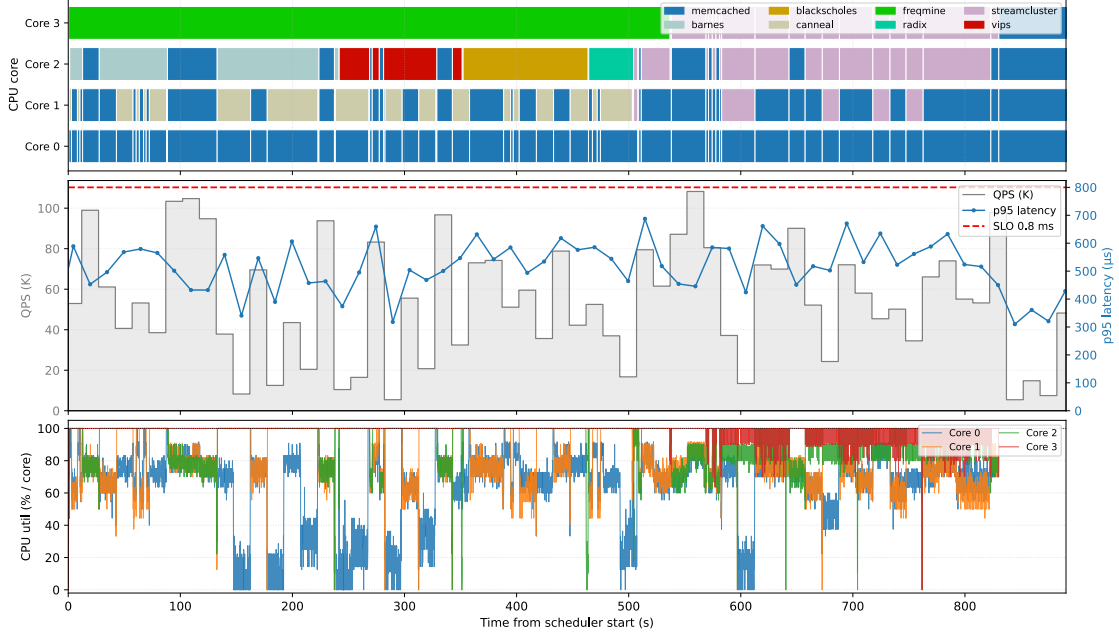


Figure 15: Part 4 Q3 Run 2, adaptive scheduler execution with interval=15 s. The controller achieves 0/56 SLO violations during the batch execution window, and 0/68 over the whole measurement trace. During the batch window, max p95 = 687.5 μ s and mean p95 = 526.4 μ s, remaining below the 0.8 ms threshold. The batch makespan is 829.8 s, and peak CPU utilization reaches 100% on all four cores during the heaviest batch phases.

313 intervals in the batch execution window, giving:

$$\frac{5}{313} = 1.60\%.$$

Thus, the policy remains below the required 3% SLO violation threshold even under the higher load variability of the 5 second interval trace.

The policy remains well within acceptable bounds despite the increased load variability, suggesting the policy is versatile and resilient.

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?

Answer: The smallest `qps_interval` used while keeping the memcached SLO violation ratio under 3% is 3 s. In the `qps_interval=3` run, all seven batch jobs completed normally. During the batch execution window, there were 8 violations over 599 total measurement points:

$$\frac{8}{599} = 1.34\%.$$

This is safely below the 3% threshold, so `qps_interval=3` is the minimum interval selected

Part 4 - Q3 Run 3 with interval=15s

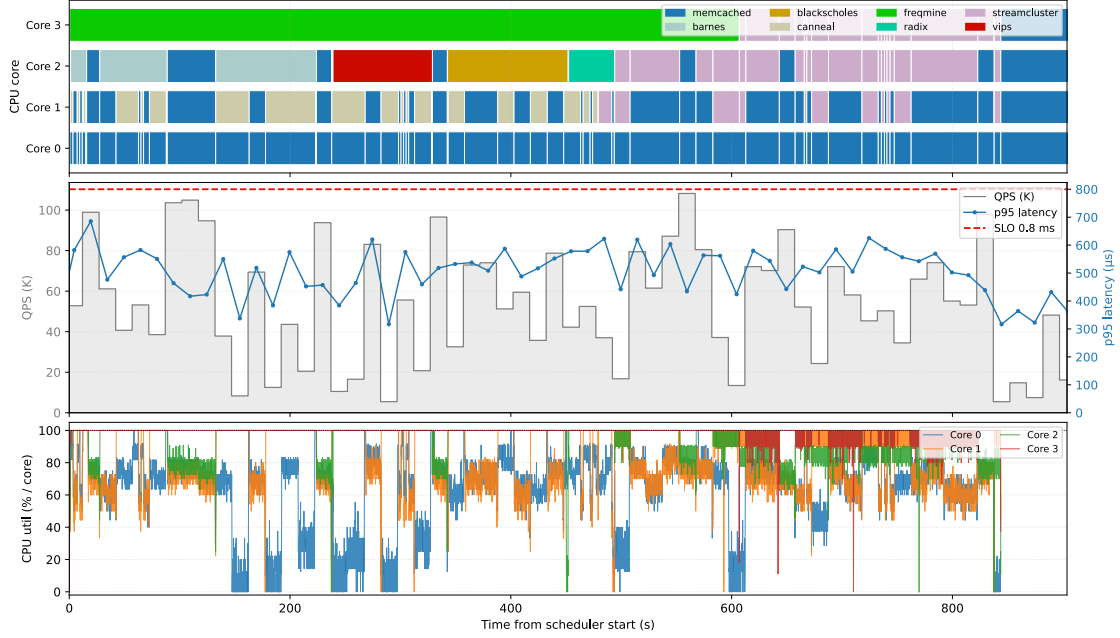


Figure 16: Part 4 Q3 Run 3, adaptive scheduler execution with interval=15 s. The controller achieves 0/56 SLO violations during the batch execution window, and 0/68 over the whole measurement trace. During the batch window, max p95 = 685.5 μ s and mean p95 = 517.7 μ s, remaining below the 0.8 ms threshold. The batch makespan is 843.4 s, and peak CPU utilization reaches 100% on all four cores during the heaviest batch phases.

for the final policy evaluation.

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.

Answer: The reasoning behind `qps_interval=3` is that it is the smallest interval where the final adaptive controller still has enough reaction time while completing all batch jobs normally. At this interval, the classifier detects a stress situation with median change spacing around 2.75 s and switches to `bounded_total_gate_slotb_plus` policy mode. This state keeps memcached between 2 and 3 cores, expands to 3 cores when total memcached CPU exceeds 175%, and shrinks back only when total CPU falls below 100%. It also pauses slot B when memcached needs core 2, keeps slot A on core 3, and throttles slot A to 10% during high total CPU periods. These features make the controller much slower than `core_fast` mode, but they absorb fast load changes well enough to keep the SLO violation ratio at 1.34% for `qps_interval=3`.

Use of AI Tools: Did you use any AI tools for this project? If so, disclose them here and explain what you used the tool(s) for and why. Note that the use of AI tools is NOT needed and NOT expected for the project.

The use of AI tools was limited to assist with automatic scripting, debugging and plotting routines: the design and arguments are authors' original. The authors take full responsibility of the code and peer-reviewed every suggestion.